



Docker Container Registries

Understanding Docker/Container Registries

- Docker/Container registries serve as repositories for storing, managing, and deploying container images. They act as centralized hubs where developers can upload, share, and retrieve container images.
- Docker/Container registries play a crucial role in container management by providing a centralized location for storing images. They facilitate efficient deployment of applications by ensuring that the required images are readily available. Additionally, registries help in version control, dependency management, and ensuring consistency across different environments.
- When choosing a Docker/Container registry, several factors should be considered. These include security measures to protect sensitive data, scalability to accommodate growing infrastructure needs, and integration capabilities with existing DevOps tools and workflows. By evaluating these criteria, organizations can select a registry that best aligns with their requirements and supports their DevOps practices effectively.

The Top 9 Picks

- Amazon Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- Docker Hub Container Registry
- GitHub Package Registry
- GitLab Container Registry
- Google Artifact Registry (GAR)
- Harbor Container Registry
- Red Hat Quay
- Sonatype Nexus Repository OSS

Amazon Elastic Container Registry (ECR)

Amazon Elastic Container Registry (ECR) is a fully managed Docker container registry that makes it easy for developers to store, manage, and deploy Docker container images. It is integrated seamlessly with Amazon ECS (Elastic Container Service) and EKS (Elastic Kubernetes Service), providing a reliable and secure way to handle container management and deployment.

Features:

- **Automated Image Scanning:** ECR automatically scans your Docker images for vulnerabilities using the Common Vulnerabilities and Exposures (CVEs) database. This feature helps in maintaining security by identifying potential threats before deployment.
- **Image Replication Across Regions:** This feature allows you to automatically replicate container images across multiple AWS regions. It ensures faster image download times and improved availability by geographically distributing images closer to your application's runtime environment.
- **Private Repositories:** ECR provides the capability to create private Docker repositories that ensure your container images are not publicly accessible. This enhances security by giving you control over who can pull and manage your images.

Amazon Elastic Container Registry Pricing

- As a new Amazon ECR customer, you get 500 MB per month of storage for your private repositories for one year as part of the AWS Free Tier.
- Both new and existing customers get 50 GB per month of always-free storage for their public repositories. You can anonymously (without using an AWS account) transfer 500 GB of data to the Internet from a public repository each month for free. If you sign up for an AWS account, or authenticate to Amazon ECR with an existing AWS account, you can transfer 5 TB of data to the Internet from a public repository each month for free. You also get unlimited bandwidth at no cost when transferring data from a public repository to AWS compute resources in any AWS Region.
- Your free usage is calculated each month across all regions and automatically applied to your bill; free usage does not accumulate.

COMPUTE

Free Tier

12 MONTHS FREE

Amazon Elastic Container
Registry

50 GB

of storage for public repositories

Store and retrieve Docker images.

500 MB of storage for private repositories

5 TB of authenticated data transfer out of
public repositories

500 GB of anonymous data transfer out of
public repositories



Azure Container Registry (ACR)

Azure Container Registry (ACR) is a managed Docker registry service provided by Microsoft Azure that allows you to store and manage container images across all types of Azure deployments. It supports standard Docker CLI commands and works seamlessly with Azure Kubernetes Service (AKS).

Features

- **Automated Builds & Webhooks:** ACR supports automated container image builds from source code repositories (using Azure DevOps or GitHub) and notifications through webhooks, which trigger actions on events like pushing or deleting an image.
- **Task Automation:** ACR Tasks can automate Docker builds, image patching, and testing across multiple environments. This is useful for continuous integration and deployment pipelines.
- **Geo-Replication:** ACR Premium offers geo-replication capabilities, allowing you to maintain multiple registry instances across global Azure regions. This enhances performance by placing the registry closer to your users and ensuring higher availability.
- **Security and Compliance:** Integrates with Azure Active Directory for authentication, uses Azure role-based access control for permissions, and provides compliance certifications such as ISO, SOC, and PCI.

Azure Container Registry Pricing

- **Basic Tier:** Ideal for small scale deployments, offers 10 GB of storage and costs approximately \$5 per month.
- **Standard Tier:** Aimed at medium-sized deployments, provides 100 GB of storage and costs about \$20 per month.
- **Premium Tier:** Designed for large-scale and business-critical applications, includes 500 GB of storage and features like geo-replication, costing around \$50 per month.
- **Free Tier:**



Container Registry

Store and manage container images across all types of Azure deployments. Containers

1 Standard tier registry with 100 GB storage and 10 webhooks

12 months

Docker Hub Container Registry

Docker Hub is the world's largest library and community for container images, offering a centralized resource for container image discovery, distribution, and change management. It integrates well with Docker Engine, providing an effortless way for developers to publish and exchange container images.

Features:

- **Vast Repository:** Docker Hub hosts millions of public and private repositories, allowing users to collaborate across teams and the broader Docker community. Users can find official images from software vendors and certified content.
- **Automated Builds:** Supports automated building and testing of container images from GitHub and Bitbucket repositories upon code commit, ensuring updated images are always available.
- **Webhooks and Notifications:** Allows for the setup of webhooks, which trigger notifications or actions when specific events occur within the registry, such as updates or pushes.
- **Integration and Compatibility:** Seamlessly integrates with the Docker ecosystem, providing a smooth workflow for Docker Engine users. It's compatible with all Docker Compose and Docker Swarm features.

Docker Hub Container Registry Pricing

- **Free Plan:** Provides one private repository and unlimited public repositories. Automated builds are available with some limitations on build times and parallel builds.
- **Pro Plan:** At \$7 per month, this plan offers unlimited private repositories, higher priority for automated builds, and no rate limiting on image pulls.
- **Team Plan:** Aimed at teams and organizations, priced at \$11 per user per month, this plan includes advanced team management features, more parallel builds, and enhanced collaboration tools.

Personal Ideal for individual developers and students. \$0 ✓ Docker Desktop ⓘ ✓ Unlimited public repositories ✓ Docker Engine + Kubernetes ⓘ ✓ 200 image pulls per 6 hours ✓ Unlimited scoped tokens ⓘ Start now	Pro Monthly Includes pro tools for individual developers who want to accelerate their productivity. \$7 per month Everything in Personal plus: ✓ Unlimited private repositories ✓ 5,000 image pulls per day ✓ 5 concurrent builds ⓘ ✓ 300 Hub vulnerability scans Buy now	Team Monthly For smaller teams requiring collaboration and productivity tools. \$11 per user* per month Everything in Pro plus: ✓ Up to 100 users ✓ Unlimited teams ✓ 15 concurrent builds ⓘ ✓ Unlimited Hub vulnerability scans ✓ Add users in bulk ✓ Audit logs ⓘ *Minimum of 5 seats, billed monthly for \$35. \$11/user/month for additional seats. Buy now	Business Yearly Ideal for businesses looking for centralized management and advanced security capabilities. \$24 per user* per month Everything in Team plus: ✓ Unlimited users ✓ Hardened Docker Desktop ⓘ ✓ Centralized management ✓ Registry Access Management ✓ Single Sign-On (SSO) ✓ SCIM user provisioning ✓ VDI support ✓ Purchase via invoice Only available with an annual subscription *Minimum 5 seats Contact Sales Buy now
--	---	---	--

GitHub Package Registry

GitHub Package Registry is an advanced package management service that allows GitHub users to publish, manage, and install software packages directly alongside their source code. It supports several package formats including Docker containers, ensuring a seamless integration with GitHub workflows and CI/CD pipelines.

Features

- **Integrated with GitHub:** Direct integration with GitHub repositories. Use the same GitHub interface for source code and package management, simplifying access control and reducing the complexity of managing project dependencies.
- **Support for Multiple Package Formats:** Besides Docker images, it supports npm, NuGet, Maven, Gradle, and RubyGems, making it versatile for projects in different programming languages and environments.
- **CI/CD Integration:** Easily integrates with GitHub Actions, allowing for automatic package publishing and versioning directly from GitHub workflows.
- **Visibility and Access Control:** Offers fine-grained permissions tied to GitHub user accounts and roles. Packages can be made public for open-source projects or private for internal use, aligning with repository visibility settings.

GitHub Package Registry Pricing

Public Repositories:

- Free for all users.
- Unlimited public package downloads and publishing at no cost.

Private Repositories:

GitHub offers different storage and data transfer allowances depending on the user's subscription plan.

- **Free Plan:** Includes 500 MB of storage and 1 GB per month of data transfer outside of GitHub Actions, with unlimited data transfer within GitHub Actions.
- **Team Plan:** Provides 2 GB of storage and 10 GB per month of data transfer outside of GitHub Actions, also with unlimited data transfer within GitHub Actions.
- **Enterprise Plan:** Offers 50 GB of storage and 100 GB per month of data transfer outside of GitHub Actions, plus unlimited data transfer within GitHub Actions.

Private repositories

Plan	Storage	Data transfer out within Actions	Data transfer out outside of Actions	
Free	500MB	Unlimited	1GB per month	Included in Pro
Team	2GB	Unlimited	10GB per month	Most Popular Continue with Team
Enterprise	50GB	Unlimited	100GB per month	Start a Free Trial Contact Sales

GitHub Packages is not available for private repos in legacy per-repository plans.

Additional storage

\$0.25 USD
per gigabyte

Additional data transfer out
outside of Actions

\$0.50 USD
per gigabyte

GitLab Container Registry

GitLab Container Registry is an integrated part of the GitLab ecosystem, providing a secure and private registry for Docker images. This registry is **free** and fully integrated with GitLab CI/CD, making it a seamless option for building, testing, and deploying Docker containers within GitLab projects.

Features

- **Seamless Integration with GitLab:** Users can easily build, push, and pull images to/from the registry within GitLab CI/CD pipelines, leveraging GitLab's single application experience.
- **Security and Compliance:** Offers robust security features such as Container Scanning, Dependency Scanning, and integration with GitLab's security dashboard for comprehensive vulnerability management.
- **Efficient Image Storage:** Uses Docker's distribution-based image storage to reduce duplication and save space, making it more efficient.
- **Visibility and Permissions Management:** Provides fine-grained access controls tied to GitLab user permissions, ensuring that only authorized users can access and manage the images.

Google Artifact Registry

Google Artifact Registry is a single, secure repository for managing container images and language packages (such as Maven and npm) across all Google Cloud services. It provides a fully managed service that scales to meet your storage needs and integrates deeply with Google Cloud's security and compliance features.

Features

- **Support for Multiple Artifact Formats:** Besides Docker container images, GAR also supports Maven and npm packages, making it a versatile choice for storing different types of artifacts in one place.
- **Integration with Google Cloud Services:** Seamlessly integrates with Google Cloud Build, Kubernetes Engine, and other Google Cloud services for continuous integration and delivery.
- **Fine-Grained Access Control:** Utilizes Google Cloud's Identity and Access Management (IAM) for detailed access control, allowing you to specify who can upload, view, or delete artifacts.
- **Security and Compliance:** Offers built-in vulnerability scanning for containers and automated policy enforcement to ensure compliance with security standards.

Harbor Container Registry

Harbor is an open-source **free** container registry that enables secure storage and management of container images. It is designed to be used in environments that require high levels of security and supports scanning images for vulnerabilities and managing user access.

Features

- **Security and Compliance:** Harbor includes robust features like vulnerability scanning, image signing with Notary, and policy-driven image replication. These features ensure that the images are secure and comply with regulatory standards.
- **Role-Based Access Control (RBAC):** Allows fine-grained access control for users and teams, ensuring that only authorized personnel can access specific images.
- **Multitenancy Support:** Harbor supports multiple users or teams (projects) to manage and segregate their container images within a single registry, enhancing organizational control and efficiency.
- **Replication:** Enables images to be replicated between multiple instances of Harbor and other registries, supporting disaster recovery and increasing the availability of images across different geographies.

Sonatype Nexus Repository OSS

Sonatype Nexus Repository OSS (Open Source Software) is a powerful **free** open-source repository manager that enables you to store, organize, and manage your software artifacts. It supports a wide range of package formats and is widely used for both private and open-source applications.

Features:

- **Support for Multiple Formats:** Nexus OSS supports a variety of package formats including Docker, Maven, npm, NuGet, PyPI, RubyGems, and more, making it highly versatile for development teams using different technology stacks.
- **Component Intelligence:** Integrates with the Sonatype OSS Index, providing valuable information on vulnerabilities within the components stored in the repository.
- **Extensible:** Offers the ability to extend its capabilities through plugins, enhancing its functionality and integrating with existing development workflows.
- **Efficient Searching and Browsing:** Features a rich search interface that allows developers to quickly find and retrieve artifacts stored in the repository.



Docker “registry” project



What it is

The Registry is a stateless, highly scalable server side application that stores and lets you distribute Docker images. The Registry is open-source, under the permissive Apache license.

Why use it

You should use the Registry if you want to:

- tightly control where your images are being stored
- fully own your images distribution pipeline
- integrate image storage and distribution tightly into your in-house development workflow

Alternatives

Users looking for a zero maintenance, ready-to-go solution are encouraged to head-over to the [Docker Hub](#), which provides a free-to-use, hosted Registry, plus additional features (organization accounts, automated builds, and more).

Requirements

The Registry is compatible with Docker engine version 1.6.0 or higher.

Basic commands

Start your registry

```
docker run -d -p 5000:5000 --name registry registry:2
```

Pull (or build) some image from the hub

```
docker pull ubuntu
```

Tag the image so that it points to your registry

```
docker image tag ubuntu localhost:5000/myfirstimage
```

Push it

```
docker push localhost:5000/myfirstimage
```

Pull it back

```
docker pull localhost:5000/myfirstimage
```

Now stop your registry and remove all data

```
docker container stop registry && docker container rm -v registry
```

About Registry

A registry is a storage and content delivery system, holding named Docker images, available in different

```
Example: the image distribution/registry , with tags 2.0 and 2.1 .
```

tagged versions. Users interact with a registry by using docker push and pull commands.

```
Example: docker pull registry-1.docker.io/distribution/registry:2.1 .
```

Storage itself is delegated to drivers. The default storage driver is the local posix filesystem, which is suitable for development or small deployments. Additional cloud-based storage drivers like S3, Microsoft Azure, OpenStack Swift, and Aliyun OSS are also supported. People looking into using other storage backends may do so by writing their own driver implementing the Storage API.

About Registry

Since securing access to your hosted images is paramount, the Registry natively supports TLS and basic authentication.

The Registry GitHub repository includes additional information about advanced authentication and authorization methods. Only very large or public deployments are expected to extend the Registry in this way.

Finally, the Registry ships with a robust notification system, calling webhooks in response to activity, and both extensive logging and reporting, mostly useful for large installations that want to collect metrics.

Understanding image naming

Image names as used in typical docker commands reflect their origin:

- **docker pull ubuntu** instructs docker to pull an image named **ubuntu** from the official Docker Hub. This is simply a shortcut for the longer **docker pull docker.io/library/ubuntu** command
- **docker pull myregistrydomain:port/foo/bar** instructs docker to contact the registry located at **myregistrydomain:port** to find the image **foo/bar**

Use cases

Running your own Registry is a great solution to integrate with and complement your CI/CD system. In a typical workflow, a commit to your source revision control system would trigger a build on your CI system, which would then push a new image to your Registry if the build is successful. A notification from the Registry would then trigger a deployment on a staging environment, or notify other systems that a new image is available.

It's also an essential component if you want to quickly deploy a new image over a large cluster of machines.

Finally, it's the best way to distribute images inside an isolated network.

Requirements

You absolutely need to be familiar with Docker, specifically with regard to pushing and pulling images. You must understand the difference between the daemon and the cli, and at least grasp basic concepts about networking.

Also, while just starting a registry is fairly easy, operating it in a production environment requires operational skills, just like any other service. You are expected to be familiar with systems availability and scalability, logging and log processing, systems monitoring, and security 101. Strong understanding of http and overall network communications, plus familiarity with golang are certainly useful as well for advanced operations or hacking.

Deploy a registry server

Before you can deploy a registry, you need to install Docker on the host. A registry is an instance of the registry image, and runs within Docker.

This topic provides basic information about deploying and configuring a registry. For an exhaustive list of configuration options, see the configuration reference.

Run a local registry

Use a command like the following to start the registry container:

```
$ docker run -d -p 5000:5000 --restart=always --name registry registry:2
```

The registry is now ready to use.

Copy an image from Docker Hub to your registry

You can pull an image from Docker Hub and push it to your registry. The following example pulls the **ubuntu:16.04** image from Docker Hub and re-tags it as **my-ubuntu**, then pushes it to the local registry. Finally, the **ubuntu:16.04** and **my-ubuntu** images are deleted locally and the **my-ubuntu** image is pulled from the local registry.

1. Pull the ubuntu:16.04 image from Docker Hub.
2. Tag the image as **localhost:5000/my-ubuntu**. This creates an additional tag for the existing image. When the first part of the tag is a hostname and port, Docker interprets this as the location of a registry, when pushing.

```
$ docker pull ubuntu:16.04
```

```
$ docker tag ubuntu:16.04 localhost:5000/my-ubuntu
```

Copy an image from Docker Hub to your registry

3. Push the image to the local registry running at localhost:5000:

```
$ docker push localhost:5000/my-ubuntu
```

4. Remove the locally-cached **ubuntu:16.04** and **localhost:5000/my-ubuntu** images, so that you can test pulling the image from your registry. This does not remove the **localhost:5000/my-ubuntu** image from your registry.

```
$ docker image remove ubuntu:16.04  
$ docker image remove localhost:5000/my-ubuntu
```

Copy an image from Docker Hub to your registry

5. Pull the **localhost:5000/my-ubuntu** image from your local registry.

```
$ docker pull localhost:5000/my-ubuntu
```

Stop a local registry

To stop the registry, use the same docker container stop command as with any other container.

```
$ docker container stop registry
```

To remove the container, use docker container rm.

```
$ docker container stop registry && docker container rm -v registry
```

Basic configuration

To configure the container, you can pass additional or modified options to the docker run command.

Start the registry automatically

If you want to use the registry as part of your permanent infrastructure, you should set it to restart automatically when Docker restarts or if it exits. This example uses the `--restart always` flag to set a restart policy for the registry.

```
$ docker run -d \  
  -p 5000:5000 \  
  --restart=always \  
  --name registry \  
  registry:2
```

Customize the published port

If you are already using port 5000, or you want to run multiple local registries to separate areas of concern, you can customize the registry's port settings. This example runs the registry on port 5001 and also names it **registry-test**. Remember, the first part of the **-p** value is the host port and the second part is the port within the container. Within the container, the registry listens on port **5000** by default.

```
$ docker run -d \  
  -p 5001:5000 \  
  --name registry-test \  
  registry:2
```

Customize the published port

If you want to change the port the registry listens on within the container, you can use the environment variable **REGISTRY_HTTP_ADDR** to change it. This command causes the registry to listen on port 5001 within the container:

```
$ docker run -d \  
-e REGISTRY_HTTP_ADDR=0.0.0.0:5001 \  
-p 5001:5001 \  
--name registry-test \  
registry:2
```


Storage customization: Customize the storage location

By default, your registry data is persisted as a docker volume on the host filesystem. If you want to store your registry contents at a specific location on your host filesystem, such as if you have an SSD or SAN mounted into a particular directory, you might decide to use a bind mount instead. A bind mount is more dependent on the filesystem layout of the Docker host, but more performant in many situations. The following example bind-mounts the host directory **/mnt/registry** into the registry container at **/var/lib/registry/**.

```
$ docker run -d \  
-p 5000:5000 \  
--restart=always \  
--name registry \  
-v /mnt/registry:/var/lib/registry \  
registry:2
```

Customize the storage back-end

By default, the registry stores its data on the local filesystem, whether you use a bind mount or a volume. You can store the registry data in an Amazon S3 bucket, Google Cloud Platform, or on another storage back-end by using storage drivers.

Configuring a registry

The Registry configuration is based on a YAML file, detailed below. While it comes with sane default values out of the box, you should review it exhaustively before moving your systems to production.

Override specific configuration options

In a typical setup where you run your Registry from the official image, you can specify a configuration variable from the environment by passing `-e` arguments to your docker run stanza or from within a Dockerfile using the `ENV` instruction.

Override specific configuration options

To override a configuration option, create an environment variable named `REGISTRY_variable` where variable is the name of the configuration option and the `_` (underscore) represents indentation levels. For example, you can configure the rootdirectory of the filesystem storage backend:

```
storage:
  filesystem:
    rootdirectory: /var/lib/registry
```

To override this value, set an environment variable like this:

```
REGISTRY_STORAGE_FILESYSTEM_ROOTDIRECTORY=/somewhere
```

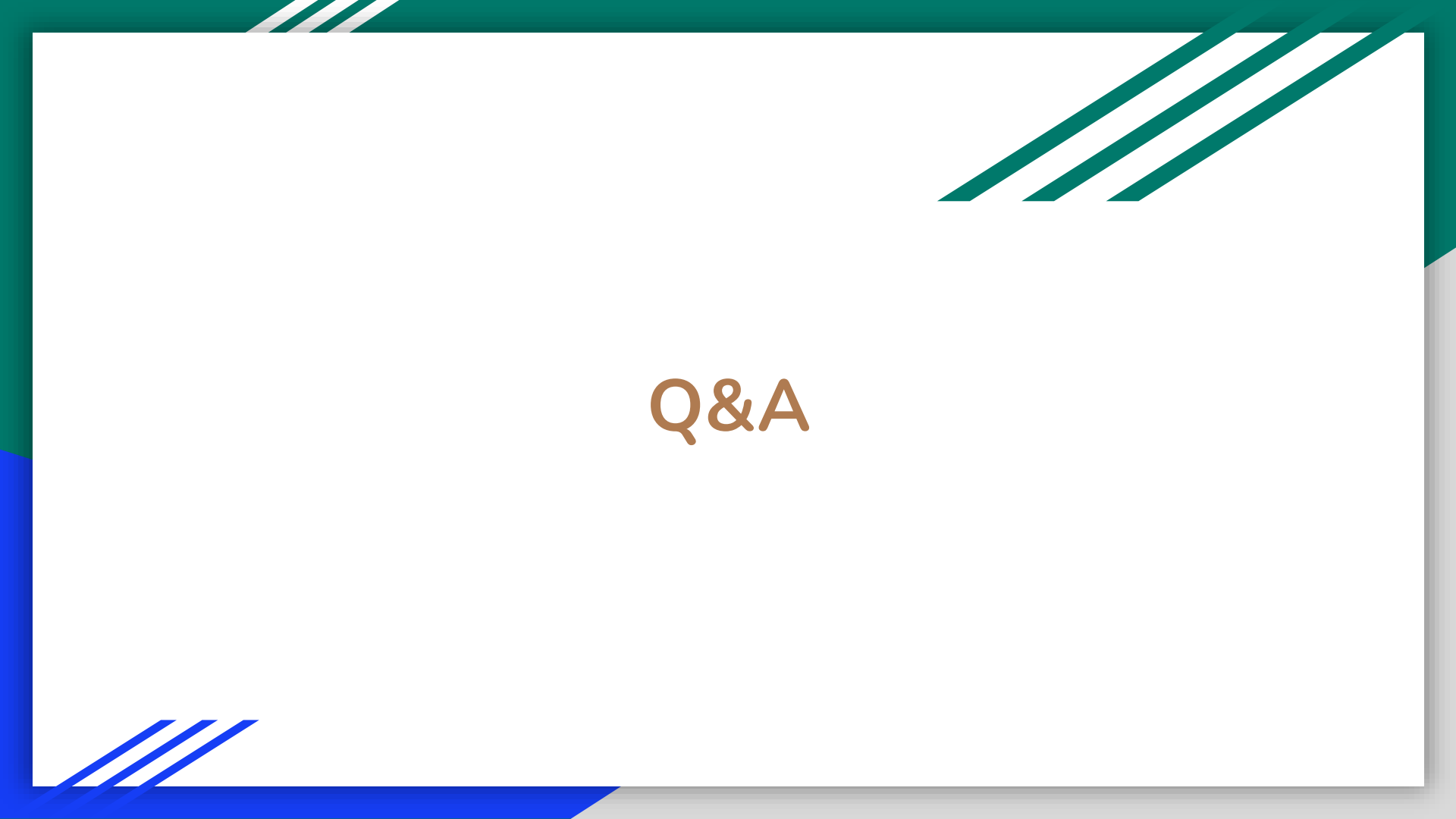
This variable overrides the `/var/lib/registry` value to the `/somewhere` directory.

Overriding the entire configuration file

If the default configuration is not a sound basis for your usage, or if you are having issues overriding keys from the environment, you can specify an alternate YAML configuration file by mounting it as a volume in the container.

Typically, create a new configuration file from scratch, named `config.yml`, then specify it in the docker run command:

```
$ docker run -d -p 5000:5000 --restart=always --name registry \
  -v `pwd`/config.yml:/etc/docker/registry/config.yml \
  registry:2
```



Q&A